

1. Introduzione

Questo documento descrive l'architettura e il design del software, evidenziando la suddivisione in moduli, il flusso dei processi e l'allocazione delle risorse. L'obiettivo è fornire una visione chiara della struttura del sistema, supportando lo sviluppo, la manutenzione e l'evoluzione della piattaforma.

2. Architettura Software

L'architettura del progetto segue il pattern **MVC (Model-View-Controller)**, suddiviso nei seguenti pacchetti:

- **Model (services):** Contiene la logica applicativa per la gestione dei dati e l'interazione con il database.
- **Controller (backend):** Gestisce le interazioni tra l'utente e il Model, fungendo da intermediario per l'elaborazione degli input.
- **View (GUI):** Interfaccia utente sviluppata con **Vaadin**, che permette la visualizzazione delle informazioni e l'interazione con il sistema.

Flusso di funzionamento MVC

L'utente interagisce con la **View**, che inoltra le richieste al **Controller**. Il **Controller** processa le richieste e comunica con il **Model** per recuperare, aggiornare o eliminare i dati dal database. Infine, il **Model** invia le informazioni aggiornate indietro al **Controller**, che le restituisce alla **View** per la visualizzazione.

Componenti e Connettori

- **Componenti:** Raccolte di procedure suddivise per funzionalità.
- **Connettori:** Chiamate di procedure, basate su un unico thread di controllo.

Punti di vista dell'architettura

- **Modulo:**
 - **Autenticazione:** Gestisce login e verifica delle credenziali.
 - **Carrello:** Logica per la gestione degli ordini e dei prodotti selezionati.
 - **Database:** Archivia dati relativi a utenti, prodotti e ordini.
 - **Interfaccia grafica:** Comunicazione tra utente e sistema.
- **Componenti e connettori:**
 - **Processi:** Comunicazione tra i vari moduli.
 - **Dati condivisi:** Organizzazione e persistenza dei dati.
- **Allocazione:**
 - **Incarico di lavoro:** Suddivisione del lavoro tra i membri del team.

- **Distribuzione:** Comunicazione tra il sistema e il database.

Questo punto di vista si concentra sulla **struttura statica** del codice sorgente e sulla sua organizzazione in moduli.

Punto di vista del Modulo (Module View)

Componenti Principali

Il software è organizzato in diversi moduli, rappresentati dalle seguenti classi:

- **Gestione Utenti**
 - *User.java* → Definisce un utente con attributi (id, nome, cognome, email, password, ruolo)User.
 - *Current.java* → Gestisce l'utente attuale permettendo una vista personalizzata in base all'utente loggato in quel momento
- **Gestione del Carrello**
 - *Cart.java* → Contiene una lista di *CartItem* per gestire gli articoli nel carrello Cart.
 - *CartItem.java* → Rappresenta un elemento nel carrello con un riferimento a un *Product* e una quantità CartItem.
- **Gestione Prodotti**
 - *Product.java* → Rappresenta un prodotto con attributi come id, taglia e modelloProduct.
 - *Model.java* → Definisce il modello di un prodotto con nome, prezzo, categoria e immagineModel.
- **Gestione Indirizzi**
 - *Address.java* → Contiene le informazioni sugli indirizzi di spedizione degli utentiAddress.

Punto di vista del Processo (Process View)

Il punto di vista del **processo** descrive il comportamento **dinamico** del sistema, concentrandosi su **flusso di dati, eventi e concorrenza**.

Flusso Principale del Sistema

1. Autenticazione e gestione utente

- L'utente accede al sistema, e *Current* tiene traccia dell'utente attuale.

2. Navigazione tra i prodotti

- L'utente può visualizzare un prodotto (*Product*) e il relativo modello (*Model*).
- *Current* memorizza la categoria attualmente visualizzata.

3. Gestione del carrello

- Un utente aggiunge un prodotto al carrello (*Cart.addItem(Product, quantity)*).
- *CartItem* viene creato e aggiunto all'elenco degli articoli nel carrello.
- L'utente può rimuovere un elemento (*Cart.removeItem(CartItem)*).
- L'utente può svuotare il carrello (*Cart.emptyCart()*).

4. Gestione dell'acquisto

- L'utente può completare l'acquisto selezionando uno dei suoi indirizzi registrati o aggiungendone uno nuovo. Successivamente, inserisce i dati di pagamento e conferma l'ordine.
- Se il pagamento viene accettato, viene creata un'istanza della classe **Order**, che registra l'acquisto nel database.

Punto di vista dell'Allocazione (Allocation View)

Questo punto di vista descrive come il software è **distribuito** su hardware o ambienti di esecuzione.

Tecnologie Coinvolte

- **Backend:** Spring Boot (con Spring Context per la gestione dei bean, es. *@Component* in *Current.javaCurrent*).
- **Database:** Utilizziamo SQLite tramite l'applicazione db Browser
- **Frontend:** Per la creazione del frontend ci siamo appoggiati a Vaadin

Diagramma dei package:

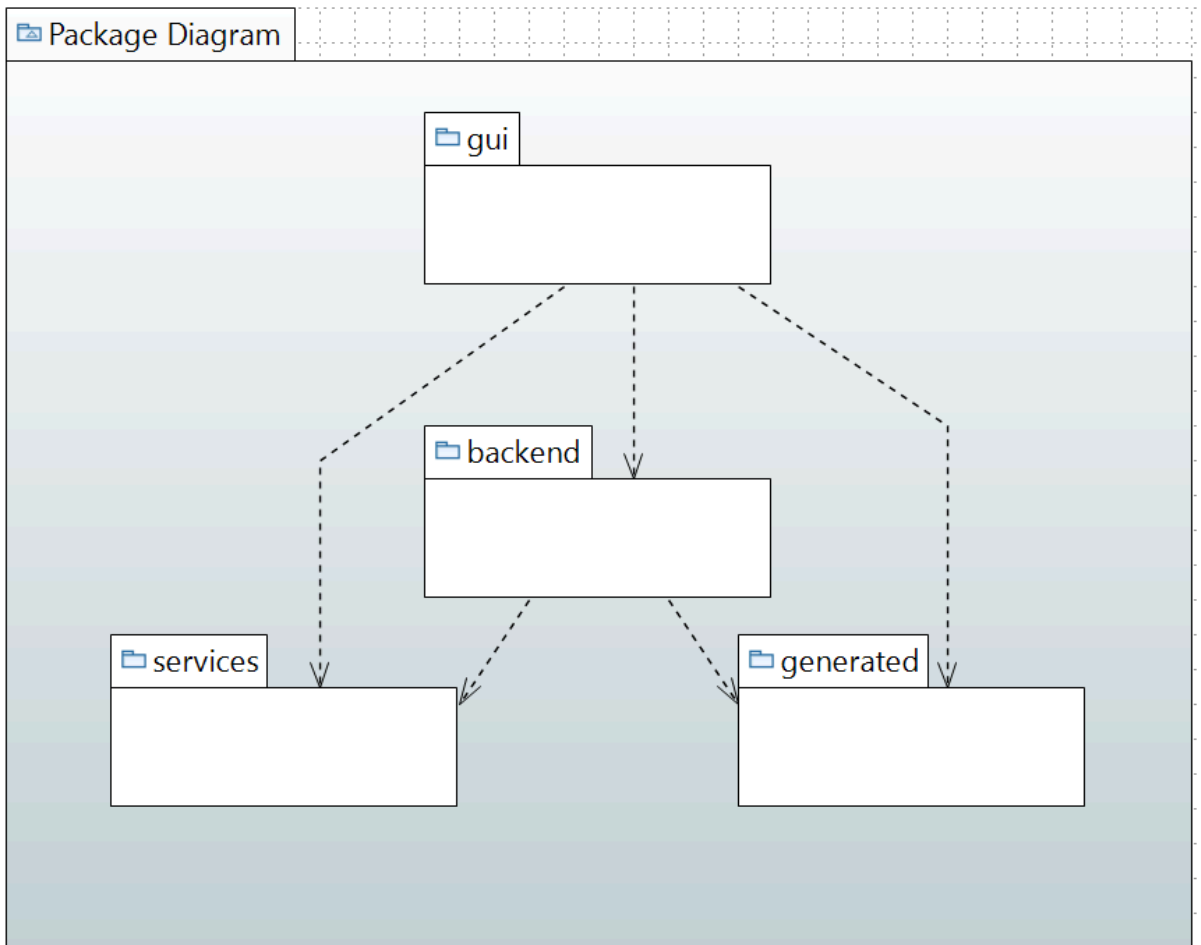
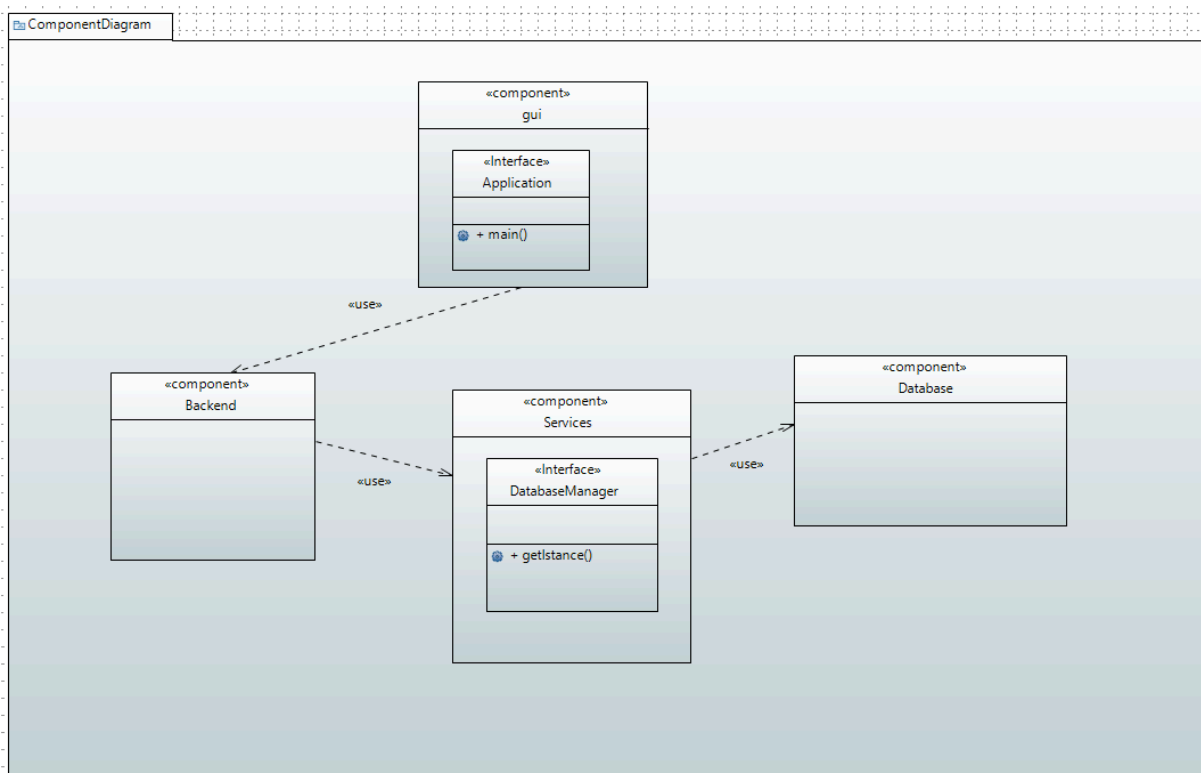


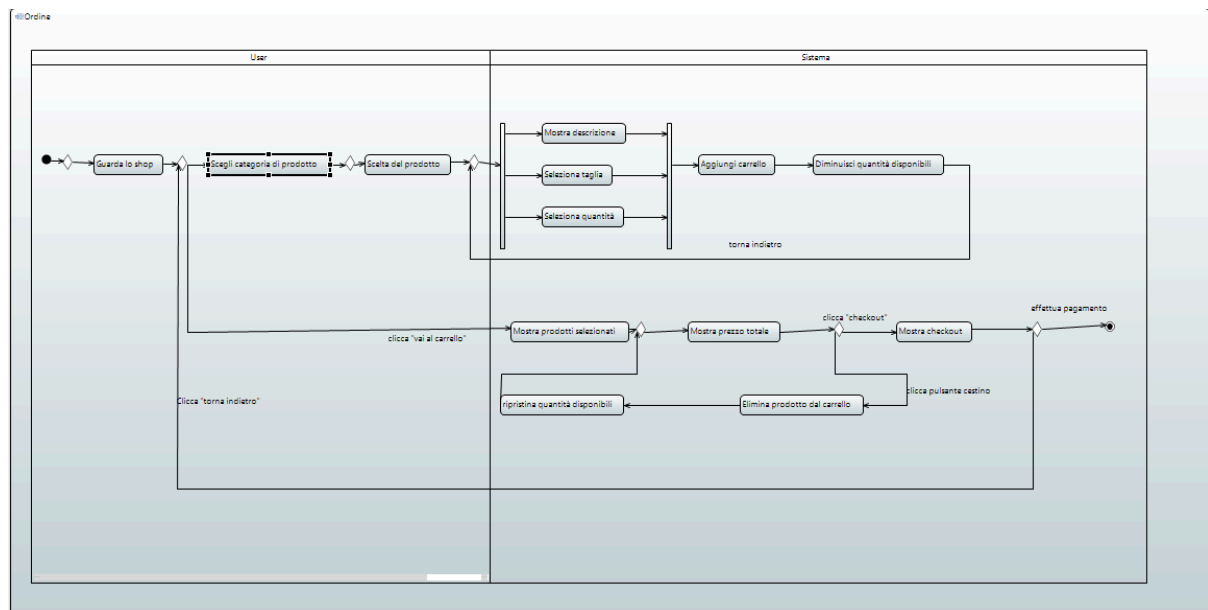
Diagramma dei componenti:



3. Librerie Utilizzate

- **Vaadin:** Framework per la creazione di interfacce web in Java.
- **Spring Boot:** Framework per lo sviluppo di applicazioni Java-based.
- **JDBC:** Connessione al database.
- **Apache Log4j:** Gestione dei log per debugging.

4. Software Design



4.1 Principi di Progettazione

Analisi Statica del Codice

L'analisi statica è stata effettuata tramite **SonarLint** e **Stan4J**, che hanno permesso di rilevare:

- **Bug:** Errori logici e difetti di implementazione.
- **Code Smells:** Problemi di design che riducono la manutenibilità del codice.
- **Vulnerabilità di sicurezza:** Identificazione di potenziali falle nel sistema.
- **Debito Tecnico:** Parti del codice che necessitano di miglioramenti.

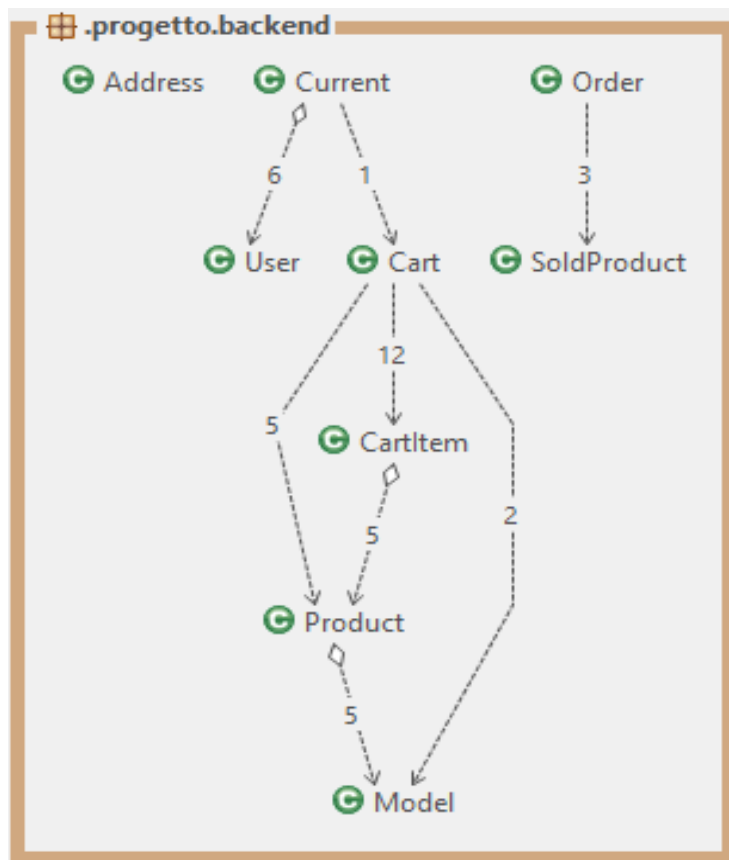
Analisi Strutturale del Codice

Complessità Ciclomatica

Utilizzando **Stan4J**, abbiamo misurato la complessità ciclomatica del codice.

Complessità ciclomatica totale è pari a 3.06.

Questo è il grafico che rappresenta le dipendenze tra le classi, interfacce ed enumerativi all'interno del package backend:



Anche altri fattori come il coupling sono importanti. Per quanto riguarda il backend, abbiamo i seguenti valori:

I	0.06
Ca	15
Ce	1

4.2 Design Patterns

Nel progetto sono stati implementati i seguenti design patterns:

1. Singleton Pattern - Utilizzato nella classe Current.

- Il metodo `getInstance()` garantisce che ci sia un'unica istanza della classe `Current`, creando un'istanza solo quando necessario (lazy initialization).
- La classe ha un costruttore privato per impedire l'istanziamento diretta.

2. **Factory Method** - Utilizzato nella classe DatabaseManager .

- La classe ConnectionFactory segue il pattern Factory Method per creare e restituire connessioni al database. Il metodo statico getConnection() incapsula la logica di creazione della connessione, nascondendo i dettagli di implementazione e configurazione.